

ECE 332 Lab 4: FPGA Acceleration of GEMM for Handwritten Digit Recognition

Overview

In previous labs, you implemented image capture, preprocessing, and CNN inference on the CPU.

In this lab, you will accelerate the **GEMM (matrix multiplication)** operation using the FPGA on the DE1-SoC board using OpenCL. You will integrate this into the inference pipeline, display the predicted digit on the 7-segment display, and measure performance improvement over the CPU.

Objectives

- Implement matrix multiplication using an OpenCL kernel
- Validate the kernel using a Jupyter notebook
- Compile the kernel to generate an .aocx file
- Program the FPGA on the DE1-SoC board
- Integrate FPGA-based GEMM into CNN inference
- Display predicted digit on 7-segment display
- Measure and compare CPU vs FPGA execution time

Prerequisites

- Completion of Lab 3
- Working image capture and preprocessing pipeline
- Basic knowledge of C/C++ and matrix multiplication

System Flow

Camera → Preprocessing → GEMM (CPU / FPGA) → CNN Output → 7-Segment Display

The GEMM operation is the main computational bottleneck. In this lab, you will replace the CPU implementation with an FPGA-based accelerator.

Part 1: OpenCL Kernel Development

In this part, you will write your own OpenCL kernel for performing matrix multiplication, and test its correctness using a Jupyter notebook.

Follow the steps below:

1. Navigate to the resource folder and locate the template file `gemm_kernel.cl`.

2. Write the OpenCL kernel code for matrix multiplication in the .cl file.
 - Follow the logic discussed in class.
 - Ensure proper indexing of rows and columns.
 - Use appropriate memory access patterns (e.g., global memory).
 - Use barriers and work-item IDs carefully if needed.
3. Save your kernel file.
4. Locate the Jupyter notebook named `kernel_testing.ipynb` in the resource folder.
5. Open the notebook on your local machine or the lab server.
6. Test your kernel using the provided notebook:
 - It will use your .cl file and compare the results against a known correct CPU implementation. **Make sure kernel name in the notebook matches with your kernel name.**
 - Review the outputs and ensure the kernel produces correct matrix multiplication results.
7. If the notebook flags any mismatches or errors, go back to your .cl file and debug your kernel logic. Rerun the notebook to test again after corrections.

Part 2: Kernel Compilation

In this part, you will compile your OpenCL kernel into an FPGA bitstream (.aocx) using the aoc compiler on a host machine.

You may choose **one** of the following options:

Option 1: Using Server (Linux Environment)

Connecting and copying files to server instructions are in Lab 0 module of course website

1. Extract the `de1_soc` folder from the provided resources.
2. Copy the contents from your local machine to the server:

```
1 scp -r Lab4RequiredFiles username@server_ipaddr:/remote/path/
```

3. Set up the OpenCL environment:

```
1 source /opt/intelFPGA-lite/hld/init_opencl.sh
```

4. Set the board package path:

```
1 export AOCL_BOARD_PACKAGE_ROOT="<path to Lab3 folder>/Lab3RequiredFiles/  
  Lab3_resoruces/de1_soc"
```

5. Verify available boards:

```
1 aoc -list-boards
```

Ensure that `de1soc_sharedonly` is listed.

Option 2: Using Lab Machines (Windows Environment)

1. Extract the de1_soc folder from the resources.

2. Initialize the OpenCL environment:

- Navigate to: C:\intelFPGA\hld\
- Run: init_opencl.bat

3. Copy the hardware folder to:

```
1 C:\intelFPGA\hld\board\al0_ref\
```

Replace the hardware folder, this is because we don't have permissions to change environment variables on the host machine. If you are working on your own system, instead of replacing the folder set the environment variable **AOCL_BOARD_PACKAGE_ROOT** to your own directory

4. Copy the tests folder to the same al0_ref directory.

5. Verify board availability:

```
1 aoc -list-boards
```

Ensure that de1soc_sharedonly is listed.

Kernel Compilation

Now compile your GEMM kernel (here assumption is that kernel name is gemm_kernel.cl):

```
1 aoc device/gemm_kernel.cl -o bin/gemm_kernel.aocx -board=de1soc_sharedonly
```

This generates the FPGA bitstream (.aocx), which contains the hardware implementation of your OpenCL kernel.

Verification (Optional)

You may test your setup by compiling an example kernel:

```
1 aoc device/vector_add.cl -o bin/vector_add.aocx -board=de1soc_sharedonly
```

This confirms that your OpenCL environment and board configuration are set up correctly.

Part 3: Programming the FPGA from ARM Linux

- Copy the .aocx file to the Board
- Set up OpenCL environment:

```
1 cd OpenCL/  
2 source init_opencl.sh
```

Program the FPGA:

```
1 aocl program /dev/acl0 bin/gemm_kernel.aocx
```

Part 4: FPGA Host Integration (GEMM Backend)

In this part, you will integrate your OpenCL kernel with the host (ARM CPU) program to execute GEMM on the FPGA.

You are provided with a partially completed function:

```
1 bool gemm_backend_run_fpga(const std::vector<float>& A,  
2                             const std::vector<float>& B,  
3                             std::vector<float>& C,  
4                             int M,  
5                             int K,  
6                             int N)
```

This function is responsible for:

- Allocating device buffers
- Transferring input data to FPGA
- Launching the OpenCL kernel
- Reading back the output

Complete the following steps:

TODO 1: Create Buffers

Create OpenCL buffers for matrices B and C using `clCreateBuffer`.

- B should be read-only
- C should be write-only

TODO 2: Write Input Data

Transfer matrix B from host to device using:

```
1 clEnqueueWriteBuffer(...)
```

TODO 3: Set Kernel Arguments

Set all required kernel arguments using `clSetKernelArg`.

Typical arguments include:

- Input buffers: A, B
- Output buffer: C
- Dimensions: M, K, N

Ensure the order matches your OpenCL kernel definition.

TODO 4: Read Output Data

After kernel execution, read the output matrix C back to host memory:

```
1 clEnqueueReadBuffer(...)
```

Execution Flow Summary

1. Create device buffers
2. Copy inputs (A, B) to FPGA
3. Set kernel arguments
4. Launch kernel
5. Wait for completion
6. Read output (C) back to host

Note: This flow is similar to GPU programming, but here the kernel is executed on FPGA hardware generated by the aoc compiler.

Capture → Preprocess → Infer → Display

Part 5: 7-Segment Display Output

- Map predicted digit (0–9) to 7-segment display
- Use memory-mapped I/O (hex base address)
- Display result after each inference

Part 6: Performance Measurement

Measure the following:

- CPU GEMM execution time
- FPGA GEMM execution time

Compute Speedup

Measure execution time for both CPU and FPGA implementations across multiple runs and layers.

- Run the experiment multiple times (at least 3–5 runs) and report the average time.
- Measure performance for different layers of the network (e.g., multiple GEMM calls).
- Record CPU time and FPGA time for each case.

Compute speedup using:

$$\text{Speedup} = \frac{\text{CPU Time}}{\text{FPGA Time}}$$

Report:

- Per-layer timing comparison (CPU vs FPGA)
- Average timing across runs
- Observed speedup for each layer

Deliverables

- `gemm_kernel.cl`
- Working system demo showing digit on 7-segment display
- Timing results (CPU vs FPGA)
- Lab report

Lab Report

Include the following:

- Kernel design explanation
- Debugging process
- Timing comparison (CPU vs FPGA)
- Speedup calculation

Concept Questions: OpenCL Compilation

Answer the following questions in your lab report:

1. What is the purpose of the `aoc` command? What does it take as input and what does it generate as output?
2. The `.aocx` file is used to program the FPGA. What does this file contain at a high level?
3. Does the `aoc` compilation process generate hardware? Briefly explain what happens between the OpenCL kernel (`.cl`) and the final FPGA execution.
4. Why is compiling the OpenCL kernel significantly slower than compiling a normal C/C++ program?
5. If you make a small change in your kernel code, do you need to re-run the `aoc` command? Why?